

20. Component Specification

Component inventory, states, behaviors, and accessibility rules

Program	QuantPipeline - private modular portfolio decision platform
Document purpose	Component inventory, states, behaviors, and accessibility rules
Status	Working baseline for design and delivery

This document defines the first component baseline for QuantPipeline. It is intentionally product-specific: components are described not only by appearance but by the analytical and operational jobs they must support.

1. Component system principles

- Components should encode product meaning, not just generic layout.
- Every component must define states, empty conditions, loading behavior, and error behavior.
- Keyboard and accessibility requirements are part of the component contract.
- Permissions and role differences should be handled through configuration, not ad-hoc duplication.

2. Component taxonomy

Category	Representative components	Primary responsibility
Shell and navigation	App shell, top bar, left nav, tabs, bottom nav	orientation and movement
Decision primitives	Recommendation card, confidence block, disagreement module, scenario controls	decision understanding and challenge
Data display	Metric strip, comparison table, chart container, timeline	evidence presentation
Operational controls	Queue item, approval bar, incident card, policy panel	admin and release control
Feedback	Inline alert, toast, empty state, skeleton, error panel	state communication
Inputs	search, filters, segmented controls, date range, sliders, multi-select	user control of scope and assumptions

3. Detailed component contracts

Recommendation card

Purpose: Presents one recommendation summary in overview or mobile contexts.

Required states: default, highlighted, provisional, saved, stale, unavailable

- Must define a compact variant and a full-detail variant where relevant.
- Must expose explicit status labels; color alone is insufficient.
- Must specify loading, empty, and failure behavior.
- Must define analytics events for decisive interactions.

Confidence block

Purpose: Separates model confidence, data quality, and operational readiness.

Required states: healthy, low-confidence, stale-data, degraded-ops

- Must define a compact variant and a full-detail variant where relevant.
- Must expose explicit status labels; color alone is insufficient.
- Must specify loading, empty, and failure behavior.
- Must define analytics events for decisive interactions.

Disagreement block

Purpose: Explains divergence across analytical engines.

Required states: aligned, mixed, severe divergence, unavailable

- Must define a compact variant and a full-detail variant where relevant.
- Must expose explicit status labels; color alone is insufficient.
- Must specify loading, empty, and failure behavior.
- Must define analytics events for decisive interactions.

Scenario control set

Purpose: Lets the user vary assumptions and preview deltas.

Required states: default, modified, validating, failed

- Must define a compact variant and a full-detail variant where relevant.
- Must expose explicit status labels; color alone is insufficient.
- Must specify loading, empty, and failure behavior.
- Must define analytics events for decisive interactions.

Comparison table

Purpose: Structured side-by-side view of engine outputs and dimensions.

Required states: normal, filtered, row-expanded, empty, loading

- Must define a compact variant and a full-detail variant where relevant.
- Must expose explicit status labels; color alone is insufficient.
- Must specify loading, empty, and failure behavior.
- Must define analytics events for decisive interactions.

Queue item

Purpose: Represents one publishable artifact in ops.

Required states: pending, approved, held, rolled back, escalated

- Must define a compact variant and a full-detail variant where relevant.
- Must expose explicit status labels; color alone is insufficient.
- Must specify loading, empty, and failure behavior.
- Must define analytics events for decisive interactions.

4. Shared state model for components

State layer	Rule
Default	Component has valid data and no special condition
Loading	Use skeletons or optimistic placeholders where appropriate
Empty	Explain why no content exists and what action can populate it
Degraded	Show partial data with cause and confidence implications
Error	Provide retry or investigation path
Permission-limited	Explain why action or data is unavailable to the current role

5. Accessibility requirements by component type

Component type	Accessibility rule
Navigation	clear focus order, landmark semantics, active-state identification
Cards	heading structure and readable action labels
Tables	header associations, keyboard row traversal, sticky headers
Charts	textual summary plus accessible labels for key values
Inputs	visible labels, helper text, error text, keyboard support
Alerts and toasts	severity plus readable message and dismiss behavior

6. Density and responsiveness rules

- Components should define responsive collapse behavior instead of relying on arbitrary CSS shrinkage.
- Dense desktop variants may exist, but they must still preserve readable headings and action affordances.
- Mobile variants should prioritize one primary insight and one primary action.

7. Tokenization guidance

- Expose spacing, radius, border, and elevation tokens consistently.
- Separate semantic color tokens from raw palette values.
- Text roles should use a documented token map: page title, section title, body, meta, table label.

8. Engineering and QA implications

Area	Expectation
Storybook / component preview	every major component should have state stories
Contract tests	critical state props validated by automated tests

Visual review	desktop and mobile snapshots for key states
Instrumentation	interaction events emitted consistently across variants

9. Acceptance checklist

Question	Pass condition
Does each critical component define all important states?	yes
Are accessibility rules part of the spec, not optional notes?	yes
Can admin variants reuse the same component family with role-specific behavior?	yes
Is responsiveness defined intentionally?	yes

Component system map

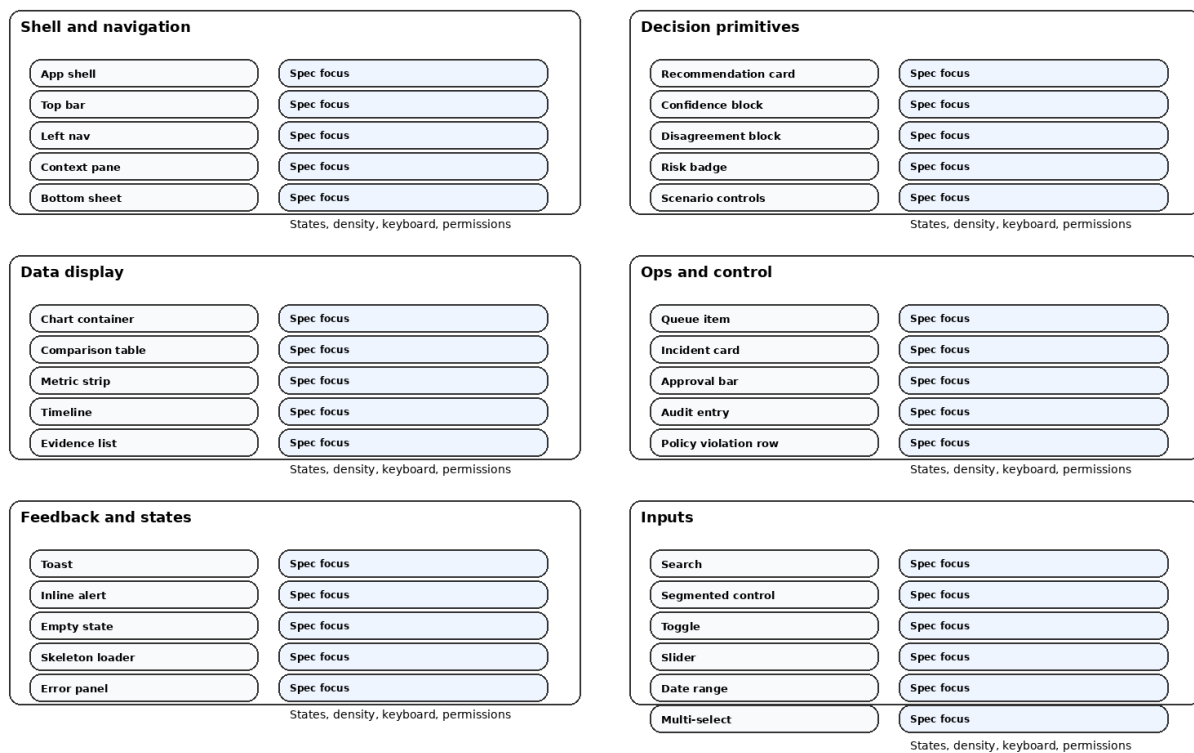


Figure 20-1. First-pass component system map.

10. Component-to-data contract expectations

Component	Minimum data contract	Failure fallback
Recommendation card	id, label, stance, weight, horizon, confidence trio	show unavailable state with explanation
Comparison table	row ids, metric schema, display order, provenance summary	show partial matrix and identify missing rows
Risk badge / strip	current exposure, threshold, status	render textual status when visuals are

	label	unavailable
Queue item	artifact id, state, actor rights, timestamps	disable actions and surface retrieval error

11. Build priority and release staging

Priority tier	Components
Tier 1 for first release	app shell, recommendation card, confidence block, comparison table, alert system, queue item
Tier 2 after baseline	scenario controls, timeline, policy panel, incident card, dense tables
Tier 3 after adoption learning	advanced chart overlays, command palette, richer watchlist management

12. Component sign-off checklist

- The component name, purpose, states, and accessibility rules are documented in one place.
- A frontend implementer can tell which props are required and which are optional.
- QA can derive visual, functional, and role-aware test cases from the spec without product guesswork.